

OSPF TTL SECURITY CHECK



OSPF TTL Security Check Explained: Operation, Example, and Configuration



OSPF is a reliable and scalable dynamic routing protocol. However, the

protocol has some weaknesses that may get exploited by attackers.

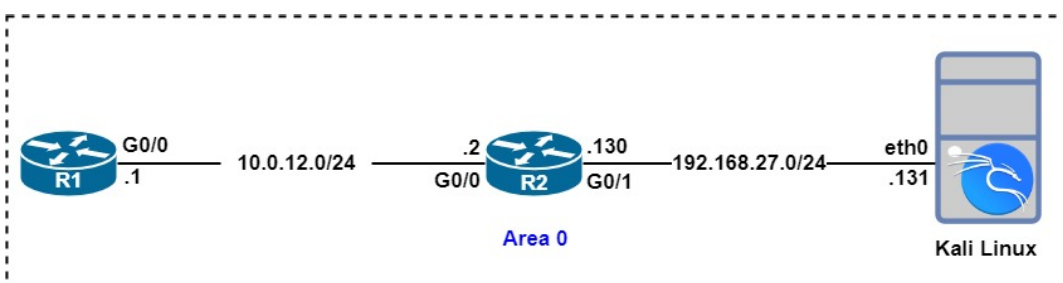
For example, hackers can run remote CPU-utilization-based attacks,

which can disrupt OSPF adjacencies. Here comes OSPF TTL Security

Check that can help protect against those kinds of attacks by allowing the processing of OSPF packets with particular TTL values.

In this tutorial, you will learn what OSPF TTL Security Check is, how this feature work using a sample attack, and how to set it up.

In the rest of this guide, we use the following network topology. It consists of two routers and a Kali Linux virtual machine from which we launch attacks against R1's OSPF process.



Here are the initial configurations of the routers.

Router R1

```
Router(config)# hostname R1
R1(config)#
R1(config)# interface GigabitEthernet0/0
R1(config-if)# ip address 10.0.12.1 255.255.255.0
R1(config-if)# ip ospf 1 area 0
R1(config-if)# no shutdown
```

Router R2

```
Router(config)# hostname R2
R2(config)#
R2(config)# interface GigabitEthernet0/0
R2(config-if)# ip address 10.0.12.2 255.255.255.0
R2(config-if)# ip ospf 1 area 0
R2(config-if)# no shutdown
R2(config-if)#
R2(config-if)# interface GigabitEthernet0/1
R2(config-if)# ip address 192.168.27.130 255.255.255.0
R2(config-if)# ip ospf 1 area 0
R2(config-if)# no shutdown
```

Kali Linux

```
sudo ip route add 10.0.12.0/24 via 192.168.27.131
```

We launched a CPU-utilisation-based attack from the Kali Linux machine using the T50 tool. We flooded router R1 with a high volume of router LSAs using this command:



```
sudo t50 10.0.12.1 --flood --turbo --protocol ospf
```

T50 sends random router LSAs to R1. It can generate up to 1,000,000 packets per second. After a few moments, routers R1 and R2 lose their OSPF adjacency, as you can see in the next example.

```
*Nov  7 23:35:29.417: %OSPF-4-ERRRCV: Received inval
*Nov  7 23:35:34.417: %OSPF-4-ERRRCV: Received inval
*Nov  7 23:35:34.752: %OSPF-5-ADJCHG: Process 1, Nbr
*Nov  7 23:35:39.461: %OSPF-4-ERRRCV: Received inval
*Nov  7 23:35:44.556: %OSPF-4-ERRRCV: Received inval
```

Here comes OSPF TTL Security Check that help reduce the impact of attacks on OSPF.



Table of Contents

1. What is OSPF TTL Security Check and How Does it Work?

2. OSPF TTL Security Check Configuration on Cisco IOS with

Examples

2.1. Configuring OSPF TTL Security Check on All OSPF Interfaces

2.2. Configuring OSPF TTL Security Check on a Per-Interface Basis

3. Related Lessons to OSPF TTL Security Check

4. Conclusion

So, What is OSPF TTL Security Check and How Does it Work?

OSPF TTL Security Check is a technique used to protect against CPU utilization-based attacks targeting OSPF-enabled routers. When this feature is enabled, OSPF packets get sent with a TTL value of 255, and OSPF processes routing packets that have a valid TTL value only.

This way, routers differentiate between legitimate OSPF packets and



fake ones and prevent consuming high CPU cycles in managing garbage OSPF packets.

OSPF TTL Security Check Configuration on Cisco IOS with Examples

Cisco IOS allows enabling OSPF TTL Security Check either globally on an OSPF process or on a per-interface basis. When you activate this feature without specifying the maximum hop count allowed, the router sets the TTL value of generated OSPF packets to 255 and rejects received OSPF packets with TTL values greater than 1.



```
Internet Protocol Version 4, Src: 10.0.12.1, Dst: 224.0.0.5
0 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
Total Length: 76
Identification: 0x0318 (792)
000. .... = Flags: 0x0
0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 255
Protocol: OSPF IGP (89)
Header Checksum: 0xc17a [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.0.12.1
Destination Address: 224.0.0.5
> Open Shortest Path First
```

Configuring OSPF TTL Security Check on All OSPF Interfaces

To enable this feature on all interfaces, not including virtual links and sham links, of an OSPF instance, use the **ttl security all-interfaces** command in router configuration mode

This example enables TTL Security Check on all OSPF-enabled interfaces on R1.



```
R1(config)# router ospf 1
R1(config-router)# ttl-security all-interfaces
```

To verify this configuration, we use the `show ip ospf` command, as you can see below.

```
R1# show ip ospf
```

omitted output

Strict TTL checking enabled

Area BACKBONE(0)

Number of interfaces in this area is 1

Area has no authentication

SPF algorithm last executed 00:01:59.024 ago

SPF algorithm executed 15 times

Area ranges are

Number of LSA 3. Checksum Sum 0x01B915

Number of opaque link LSA 0. Checksum Sum 0x000000

Number of DCbitless LSA 0

Number of indication LSA 0

Number of DoNotAge LSA 0

Flood list length 0

The **ttl security all-interfaces hops** *hop_count* command, where *hop_count* is between 1 and 254 and represents the maximum hops allowed, both activates OSPF TTL Security Check on all interfaces, except virtual links and sham links, and sets the maximum number of hops allowed to *hop_count*. In this case, the router assigns the TTL value of generated OSPF packets to 255 and discards OSPF packets with a TTL value less than $255 - \text{hop_count}$.

Moreover, you can activate OSPF TTL Security Check on virtual links



and sham links using the **area virtual-link ttl-security** and **area sham-link ttl-security** commands, respectively.

This example instructs router R2 to enable TTL Security Check on all interfaces with 1 as the maximum hop count allowed.

```
R2(config)# router ospf 1
R2(config-router)# ttl-security all-interfaces hops
```

To disable this feature on a particular interface, use the **ip ospf ttl-security disable** command in interface configuration mode.

Configuring OSPF TTL Security Check on a Per-Interface Basis



OS supports configuring TTL Security Check on a per-interface

The **ip ospf ttl-security** command to activate this feature on a
interface. To specify the maximum hop count allowed, use the
ip ospf ttl-security hops hop_count, where *hop_count* is a number

in 1 and 254.

This example enables TTL Security Check on R2's GigabitEthernet 0/0.

```
R2(config)# interface gigabitEthernet 0/0
R2(config-if)# ip ospf ttl-security
```

Finally, the following **show ip ospf interface** command output states that TTL Security Check has been activated successfully on the interface.

```
R2# show ip ospf interface gigabitEthernet 0/0
GigabitEthernet0/0 is up, line protocol is up
```



```

Internet Address 10.0.12.2/24, Area 0, Attached via
Process ID 1, Router ID 10.0.12.2, Network Type Broadcast
Topology-MTID      Cost      Disabled      Shutdown
          0          1          no          no
Enabled by interface config, including secondary address
Transmit Delay is 1 sec, State DR, Priority 1
Designated Router (ID) 10.0.12.2, Interface address 10.0.12.2
Backup Designated router (ID) 10.0.12.1, Interface address 10.0.12.1
Timer intervals configured, Hello 10, Dead 40, Wait
  oob-resync timeout 40
  Hello due in 00:00:09
Supports Link-local Signaling (LLS)
Cisco NSF helper support enabled
  LLS NSF helper support enabled
  Strict TTL checking enabled
  Index 1/1/1, flood queue length 0
  Next 0x0(0)/0x0(0)/0x0(0)
  Last flood scan length is 0, maximum is 2
  Last flood scan time is 0 msec, maximum is 1 msec
  Neighbor Count is 1, Adjacent neighbor count is 1
    Adjacent with neighbor 10.0.12.1 (Backup Designated)
  Suppress hello for 0 neighbor(s)

```

Related Lessons to OSPF TTL Security Check



- [OSPF](#)
- [OSPF Router ID](#)
- [OSPF Null Authentication](#)
- [OSPF Plain Text Authentication](#)
- [OSPF Default Route](#)
- [Basic OSPF Configuration Lab for CCNA](#)
- [OSPF Configuration](#)
- [OSPF Passive Interface](#)
- [OSPF Virtual Link](#)
- [OSPF Stub Area](#)
- [OSPF LSA Types](#)
- [OSPF Graceful Restart](#)
- [OSPF Totally Stubby Area](#)
- [OSPF Reference Bandwidth](#)
- [OSPF Cost](#)
- [OSPF DR/BDR Election](#)
- [OSPF Hello and Dead Interval](#)
- [OSPF Metric](#)
- [OSPF MD5 Authentication](#)
- [OSPF HMAC-SHA Cryptographic Authentication](#)
- [OSPF Multi-Area](#)
- [OSPF TTL Security Check](#)
- [OSPF Graceful Shutdown](#)
- [Route Redistribution between OSPF and RIP](#)



- [OSPF Network Types](#)
- [OSPF Totally NSSA Area](#)
- [OSPF NSSA Area](#)
- [OSPF Summarization](#)
- [OSPF Route Filtering](#)
- [OSPF Type 5 LSA Filtering](#)
- [OSPF ABR Type 3 LSA Filtering](#)
- [OSPF Prefix Suppression](#)
- [OSPF Path Selection](#)
- [OSPF LSA Throttling](#)
- [OSPF SPF Throttling](#)
- [OSPF Incremental SPF](#)
- [OSPF Non-Broadcast Network Type](#)
- [OSPF Point-to-Point Network Type](#)
- [OSPF Broadcast Network Type](#)
- [OSPF Point-to-Multipoint Network Type](#)
- [OSPF vs RIP](#)
- [OSPF LSA Group Pacing](#)
- [OSPF LSA Flood Pacing](#)
- [OSPF LSA Retransmission Pacing](#)
- [Troubleshooting OSPF Neighbor Adjacency](#)
- [Troubleshooting OSPF Route Installation](#)
- [Troubleshooting OSPF Route Advertisement](#)
- [OSPF Stub Router](#)



Conclusion

I hope this blog post helps you learn something.

Now I'd like to turn it over to you:

What did you like about this tutorial?

Or maybe you have an excellent idea that you think I need to add.

Either way, let me know by leaving a comment below right now.

Mohamed Ouamer

Mohamed Ouamer is a computer science teacher and a self-published author. He taught networking technologies and programming for more than fifteen years. While he loves to share knowledge and write, Mohamed's best passions include spending time with his family, visiting his parents, and learning new things.

in

